Dt.: Pg.: Delta

```
#include <iostream.h>

int main()
{
    int count = 0;
    char ch;
    cout << "Input text";
    while (ch != '\n')
    {
        cin.get(ch);
        cout.put(ch);
        count++;
    }
cout << "no. of char";
cout << count - 1;        //because it
                          counts new line also.
getch();
return 0;
}
```

get includes space, tab, newline (\n)
cin >> ch doesnot include newline, space, tab. It is skipped automatically.

→ Getline() and write() in C++
(reads full line at a time)

```
cin.getline(line, size);
```

this is Variable ↓ length.

eg:
```
char name[10];
cin.getline(name, 10);
```

VIKAS PANDEY

↳ will we written as

VIKAS PAN\n // this will be stored

eg:

```
cout.write(line, size);
cout.write(name, 5);
```

VIKAS PANDEY ⇒ will be read as

VIKAS

It doesnot terminate after finding new line (\n)

eg: cout.write(name, 15)

VIKAS PANDEY
GRADUATE

VIKAS PANDEY\n GR

eg:

```
#include<iostream.h>
int main()
{
    char name[20];
    cout<<" enter your name";
①   cin.getline(name, 15);          → 14 char, 1 would be \n
    cout<<"\n Name : " ;
②   cout.write(name, 10);           → show till 10 alphabet
③   cout<<"\n Name : "<<name;       → show full name
    getch();
    return 0;
}
```

Output:

① Enter your name: VIKAS PANDEY\n

② Name : VIKAS PAND

③ Name : VIKAS PANDEY

→ Exception handling & Errors in C++.

Errors can be corrected but exceptions should be handeled.

Exception

Synchronous
eg: array index, out of bound, overflow etc
↓
C++ deals with this exception

Asynchronous
eg: keyboard interruption etc.

Exception
- hit exception } first block
- throw exception }
- catch exception } second block
- handle exception }

try block
detect & throw exception.
↓
catch block
catch & handle exception.

```
try
{ throw exception;}

Catch (type of argument)
  {
  }
```

```
eg: int x = (a-b);
    try
    { if (x!=0)
        { cout << a/(a-b); }
      else
        { throw(x);
        }
    }
```

```
catch (int i)
{ cout << "In division by
          zero";
}
getch();
return 0;
}
```

Dt.:
Pg.:

Types of error.

(i) **Syntax error:**

(a) not following grammatical rules used in declaration of identifier.
(b) not declaring identifier used in program.
(c) not terminating statement by semicolon.
(d) not providing equal no. of opening & closing brackets.

can be rectified by user as it displayed while compiling program.

(ii) **Logical error**

(a) This error won't be displayed on screen. However, it will lead to display wrong result.
eg: infinite loop. This error lead to abnormal termination of program or infinite loop.

(iii) **Runtime error**.

(a) This error occur while running program by displaying message listed below.
Division by zero, overflow, underflow.

Dt.:
Pg.:
Delta

Some important questions for beginners

q: What would be output

(i)

```
int i = 0;
for ( i = 1; i<=10; ++i)
{
    cout << i;
}
```

```
int (i=1; i<=10; i++);
{ cout << i;
}
```

output : 11

↓

output : 1 2 3 4 5 6 7 8 9 10

(ii)

```
int f = 1;
int a;
for (a = 1; a <= num; ++a)
{
    f = f * a;
}
cout << f;
```

```
int (i=1; i<=10; ++i);
{
    cout << i;
}
```

output: 11

output: (factorial)

(iii)

```
int x;
double sum, term;
sum = 0;
for (x = 1; x <= 25; x += 3)
{
    term = 1/x;
    sum = sum + term;
}
cout << sum;
```

output: $1+\frac{1}{4}+\frac{1}{7}+\frac{1}{10}+\frac{1}{13}+\frac{1}{16}+\frac{1}{19}+\frac{1}{22}+\frac{1}{25}$

(iv)
```
int sum = 0, sq = 0;
int n;
n = 1;
while (n >= 1)
{
    sq = n * n;
    cout << sq;
    sum = sum + sq;
    n = n + 1;
}
cout << sum;
```

output: series of square of first 10 natural no and their sum.

(v)
```
double term = 10;
int count = 0;
do
{ cout << term;
    count = count + 1;
    term = term + 3.5;
}
while ( count < 10);
```

output: series upto 10 terms : 10 13.5 17 20.5..

(vi)
```
int i, c;
for (i=0; i<20; ++i)
{
    if (b == 0)
        break;
    else
{ c = a/b;
    cout << c; }
}
    cout << " code over";
```

// use continue instead and then again try;

(vii)
```
switch (ch)
{
 case 'a': cout << "a";
 case 'b': cout << "b";
 case 'c': cout << "c";
           break;
 case 'd': cout << "d";
           break;
 default : cout << " not a b c d";
}
```

(i) when ch = a; output: a
b
c

(ii) when input ch = c; output: c

(iii) when input ch = h; output: not a b c d

(viii)
```
int i, r;
i = 7;
r = 8;
while (i <= 10)
{ cout << r * i;
  i = i + 2;
}
```
output: 56
72

(ix)
```
int m = 100;
while (true)
{
```

```
if (m<10)
    continue;
m = m-10;
}
cout << m;
```

output: no output, infinite loop.

(x)
```
int i, j;
for (i=1; i<=4; i++)
{
  for (j=1; j<=i; j++)
      cout << i;
      cout << \n;
}
```

output:
1
2 2
3 3 3
4 4 4 4

(xi)
```
int i, j;
for (i=1; i<=4; i++)
{
  for (j=1; j<=4; j++)
   Cout << i;
   cout << \n;
}
```

output:
1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4

(xii)
```
int i, j;
for (i=1, i<=4; i++)
{
  for (j=4; j>=1; j--)
     cout << j;
     cout << \n;
}
```

output:
4 3 2 1
4 3 2
4 3
4

Some important concept on ++i and i++

++i : prefix version, value of i is incremented and value of expression is new value of i.
So, in short, first increment then assign

i++ : postfix version, value of i is incremented, but value of expression is original value of i.
So, in short, first assign and then increment

if i = 3

ex: z = i++ ; → z = 3

z = ++i ; → z = 4

```
ex: int x = 3, y, z ;
    y = x++;
    z = ++x;
    cout << x << y << z ;

    output : 5  3  5
```

```
ex: int var = 5
    cout << var++;      output : 5
    cout << ++var;      output : 7
```

```
ex: int val1 = 5, val2 = 10;
  for (int i = 1; i <= 3; i++)
  { cout << val1++ << --val2;      output: 5, 9
    cout << val2-- << ++val1;              9, 7
  }                                        7, 7
                                           7, 9
                                           9, 5
                                           5, 11
```